

<https://gorest.co.in/>

Токен получен после авторизации и вставлен в коллекцию постман.

This authorization method will be used for every request in this collection. You can override this by specifying one in the request.

Auth Type

Bearer Token

The authorization header will be automatically generated when you send the request.
Learn more about [Bearer Token](#) authorization.

Token

a13806ac27cee8d4a5c31994f

1. Получите список пользователей. Каким запросом получили список пользователей? Какой статус ответа получили? Что содержится в теле ответа? Сделайте скриншот.

Запрос: GET <https://gorest.co.in/public/v2/users>

Статус ответа: 200 OK

В теле ответа получен массив, в котором отображены данные пользователей (id, name, email, gender, status).

Go rest задание / Get users

Save

Share

GET

https://gorest.co.in/public/v2/users

Send

Params

Authorization

Headers (6)

Body

Scripts

Settings

Cookies

Query Params

Key	Value	Description
Key	Value	Description

Body

Cookies

Headers (31)

Test Results

200 OK

296 ms

1.9 KB

Save Response

JSON

Preview

Visualize

```
1  [
2    {
3      "id": 7866775,
4      "name": "Chandrabhaga Varrier",
5      "email": "chandrabhaga_varrier@casper-nader.example",
6      "gender": "male",
7      "status": "active"
8    },
9    {
10     "id": 7866773,
11     "name": "Urmila Varman",
12     "email": "varman_urmila@gleason-wiegand.test",
13     "gender": "female",
14     "status": "active"
15   },
16 ]
```

2. Модифицируйте запрос и получите страницу 12. Что содержится в теле ответа?
Сделайте скриншот

Запрос: GET <https://gorest.co.in/public/v2/users?page=12>

В теле ответа содержатся данные пользователей с конкретной страницы ресурса (12)

Go rest задание / Get page

GET <https://gorest.co.in/public/v2/users?page=12> Send

Params Authorization Headers (6) Body Scripts Settings Cookies

Query Params

Key	Value	Description
page	12	

Body Cookies Headers (31) Test Results 200 OK • 286 ms • 1.97 KB Save Response

{ } JSON Preview Visualize

```
1 [
2   {
3     "id": 7866262,
4     "name": "Ranjit Shah DDS",
5     "email": "shah_ranjit_dds@towne.test",
6     "gender": "female",
7     "status": "inactive"
8   },
9   {
10    "id": 7866261,
11    "name": "Eekalabya Iyer",
12    "email": "eekalabya_iyer@moore.test",
13    "gender": "female",
14    "status": "active"
15  },
16  {
17    "id": 7866260,
18    "name": "Daevi Desai PhD",
```

3. Какие отличия запроса в п.2 от запроса в п.1?

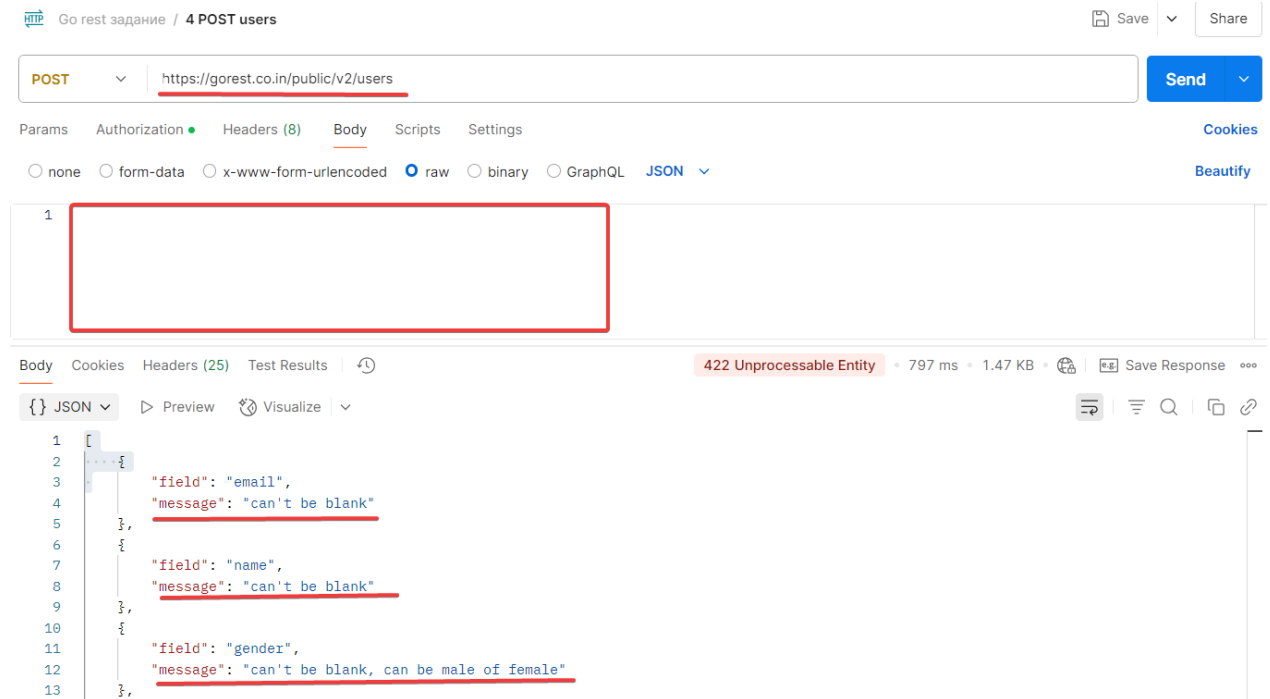
В первом запросе мы обращаемся ко всем пользователям (endpoint users)

Во втором запросе мы дополнительно передаем параметр page 12, что бы получить пользователей конкретной страницы (endpoint users?page=12)

4. Сделайте post запрос для получения пользователя с пустым телом запроса. Какой ответ пришел и почему? Сделайте скриншот

Запрос: <https://gorest.co.in/public/v2/users>

Пришел ответ с ошибкой **422Unprocessable Entity**, и перечень полей которые должны были быть заполнены в теле запроса, т.к. они обязательны для создания пользователя.



5. Напишите запрос для создания пользователя, используя метод raw => json. Какой ответ пришел? Сделайте скриншот

Запрос: <https://gorest.co.in/public/v2/users>

Тело запроса: {

```
"id": 1,  
"name": "valentine",  
"email": "valentine@casper-na3der.example",  
"gender": "male",  
"status": "active"  
}
```

Пришел ответ 201 created с параметрами которые были переданы в запросе, за исключением id (был сгенерирован уникальный), т.к. в запросе был передан уже существующий.

```
{  
  "id": 7866973,  
  "name": "valentine",  
  "email": "valentine@casper-na3der.example",  
  "gender": "male",  
  "status": "active"  
}
```

}

POST <https://gorest.co.in/public/v2/users> **Send**

Params Authorization Headers (9) **Body** Scripts Settings Cookies

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL **JSON** ☐ Beautify

```

1 {
2   "id": 1,
3   "name": "valentine",
4   "email": "valentine@casper-na3der.example",
5   "gender": "male",
6   "status": "active"
7 }

```

Body Cookies Headers (27) Test Results 201 Created • 316 ms • 1.49 KB Save Response

{} JSON Preview Visualize

```

1 {
2   "id": 7866973,
3   "name": "valentine",
4   "email": "valentine@casper-na3der.example",
5   "gender": "male",
6   "status": "active"
7 }

```

6. Напишите запрос, возвращающий в ответе запись о созданном пользователе. Сделайте скриншот.

Запрос: GET <https://gorest.co.in/public/v2/users/7866973>

GET <https://gorest.co.in/public/v2/users/7866973> **Send**

Params Authorization Headers (7) **Body** Scripts Settings Cookies

Query Params

Key	Value	Description	Bulk Edit
Key	Value	Description	

Body Cookies Headers (27) Test Results 200 OK • 903 ms • 1.43 KB Save Response

{} JSON Preview Visualize

```

1 {
2   "id": 7866973,
3   "name": "valentine",
4   "email": "valentine@casper-na3der.example",
5   "gender": "male",
6   "status": "active"
7 }

```

7. Создайте еще несколько пользователей с одинаковыми значениями "name", но разными email. Получите список всех только что созданных пользователей. Сделайте скриншот. Какими ещё get запросами можно получить запись о пользователе?

POST ▼ https://gorest.co.in/public/v2/users Send ▼

Params Authorization ● Headers (9) **Body** ● Scripts Settings Cookies

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL **JSON** ▼ Beautify

```
1 {
2   "id": 1,
3   "name": "valentine",
4   "email": "valentine@a.example",
5   "gender": "male",
6   "status": "active"
7 }
```

Body Cookies Headers (27) Test Results 🕒 **201 Created** • 368 ms • 1.48 KB • 🌐 💾 Save Response ⋮

{} JSON ▼ ▶ Preview 🔗 Visualize ▼ 🔍 📄 🔗

```
1 {
2   "id": 7866997,
3   "name": "valentine",
4   "email": "valentine@a.example",
5   "gender": "male",
6   "status": "active"
7 }
```

POST ▼ https://gorest.co.in/public/v2/users Send ▼

Params Authorization ● Headers (9) **Body** ● Scripts Settings Cookies

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL **JSON** ▼ Beautify

```
1 {
2   "id": 1,
3   "name": "valentine",
4   "email": "valentine@b.example",
5   "gender": "male",
6   "status": "active"
7 }
```

Body Cookies Headers (27) Test Results 🕒 **201 Created** • 351 ms • 1.47 KB • 🌐 💾 Save Response ⋮

{} JSON ▼ ▶ Preview 🔗 Visualize ▼ 🔍 📄 🔗

```
1 {
2   "id": 7866998,
3   "name": "valentine",
4   "email": "valentine@b.example",
5   "gender": "male",
6   "status": "active"
7 }
```

GET ▼ https://gorest.co.in/public/v2/users Send ▼

Params Authorization ● Headers (8) **Body** ● Scripts Settings Cookies

☒ none ☐ form-data ☐ x-www-form-urlencoded ☐ raw ☐ binary ☐ GraphQL

This request does not have a body

Body Cookies (1) Headers (35) Test Results 🕒 **200 OK** • 348 ms • 2.27 KB • 🌐 💾 Save Response ⋮

{} JSON ▼ ▶ Preview 🔗 Visualize ▼ 🔍 📄 🔗

```
1 [
2   {
3     "id": 7866998,
4     "name": "valentine",
5     "email": "valentine@b.example",
6     "gender": "male",
7     "status": "active"
8   },
9   {
10    "id": 7866997,
11    "name": "valentine",
12    "email": "valentine@a.example",
13    "gender": "male",
14    "status": "active"
15  }
16 ]
```

Получить данные о пользователях можно получить запросами:

GET /public/v2/users (обо всех пользователях от последнего к первому) и GET /public/v2/users/id (о конкретном пользователе указав его id)

8. Напишите запрос с методом put на изменение email и name на другое значение. Проверьте изменение get запросом. Сделайте скриншот

Запрос: PUT <https://gorest.co.in/public/v2/users/7866998>

The screenshot shows a REST client interface with a PUT request to `https://gorest.co.in/public/v2/users/7866998`. The request body is a JSON object: `{ "name": "valentinka", "email": "valentine@axc.example" }`. The response is a 200 OK status with a response time of 869 ms and a body size of 1.83 KB. The response body is a JSON object: `{ "email": "valentine@axc.example", "name": "valentinka", "id": 7866998, "gender": "male", "status": "active" }`.

```
1 {
2   |   "name": "valentinka",
3   |   "email": "valentine@axc.example"
4   | }
5
6
7
```

Body Cookies (1) Headers (28) Test Results 200 OK • 869 ms • 1.83 KB Save Response

{ } JSON Preview Visualize

```
1 {
2   |   "email": "valentine@axc.example",
3   |   "name": "valentinka",
4   |   "id": 7866998,
5   |   "gender": "male",
6   |   "status": "active"
7   | }
```

Запрос: GET <https://gorest.co.in/public/v2/users/7866998>

The screenshot shows a REST client interface with a GET request to `https://gorest.co.in/public/v2/users/7866998`. The response is a 200 OK status with a response time of 791 ms and a body size of 1.84 KB. The response body is a JSON object: `{ "id": 7866998, "name": "valentinka", "email": "valentine@axc.example", "gender": "male", "status": "active" }`. The email field is highlighted with a red box.

GET `https://gorest.co.in/public/v2/users/7866998` Send

Params Authorization Headers (8) Body Scripts Settings Cookies

Query Params

Key	Value	Description	Bulk Edit
Key	Value	Description	

Body Cookies (1) Headers (28) Test Results 200 OK • 791 ms • 1.84 KB Save Response

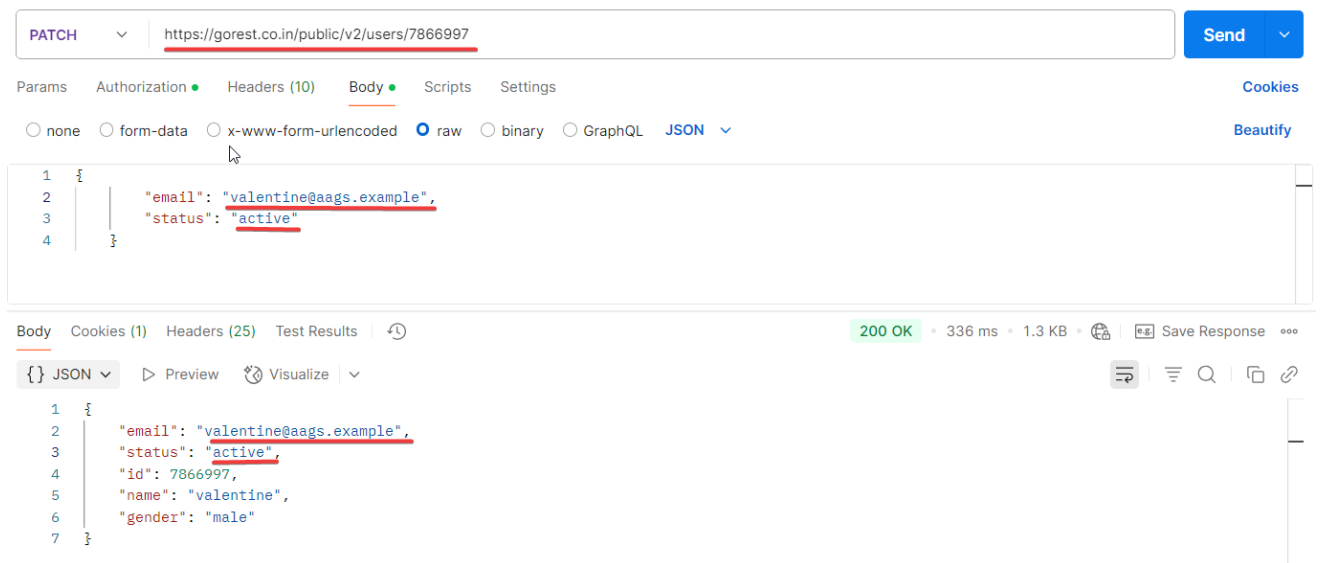
{ } JSON Preview Visualize

```
1 {
2   |   "id": 7866998,
3   |   "name": "valentinka",
4   |   "email": "valentine@axc.example",
5   |   "gender": "male",
6   |   "status": "active"
7   | }
```

9. Чем отличаются запросы put и patch? Напишите запрос с методом patch для корректировки полей status и email. Сделайте скриншот. В чем отличие полученного результата, от результата запроса из п.8?

Метод PUT – перезаписывает ресурс целиком. Метод Patch – только передаваемый компонент. Т.е. в п.8 правильно было бы передать в теле запроса все параметры. Но, в теле ответа мы получили все, несмотря на то что передали только два. Скорее всего это ввиду того, что не переданные параметры являются обязательными, и если они не передаются в теле, то не стираются как должны при использовании метода PUT, а сохраняют свои значения.

Методом Patch можно менять только конкретный параметр, сохраняя старые значения других параметров.

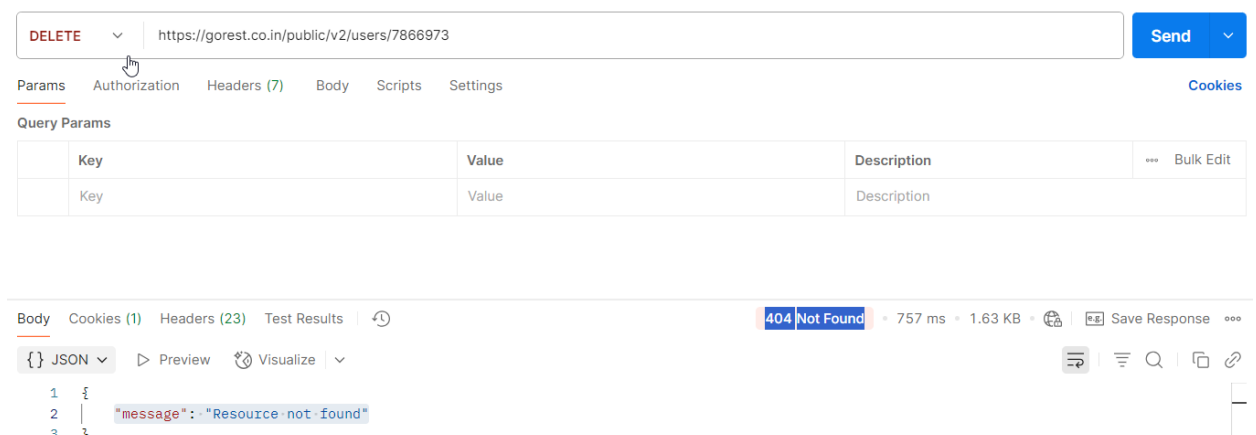


Конкретно для этой API, нет разницы в использовании методов, т.к. возвращаются все параметры.

10. Удалите пользователя, созданного в п.5. Что содержится в теле ответа? Сделайте скриншот. Выполните get запрос удаленного пользователя. Какой ответ пришел и почему? Сделайте скриншот.

Запрос: DELETE <https://gorest.co.in/public/v2/users/7866973>

Ответ: `"message": "Resource not found"` 404Not Found



Запрос: GET <https://gorest.co.in/public/v2/users/7866973>

Ответ: "message": "Resource not found" 404Not Found

GET

https://gorest.co.in/public/v2/users/7866973

Send

ParamsAuthorizationHeaders (7)BodyScriptsSettingsCookies

Query Params

	Key	Value	Description	...	Bulk Edit
	Key	Value	Description		

BodyCookies (1)Headers (23)Test Results404 Not Found850 ms1.64 KBSave Response

{ } JSON

PreviewVisualize

```
1 {
2   "message": "Resource not found"
3 }
```

Т.к. пользователь был удален, метод GET возвращает нам ответ с ошибкой, что пользователь не найден.

11. Удалите всех ранее созданных пользователей.

Запрос: DELETE <https://gorest.co.in/public/v2/users/7866998>

DELETE

https://gorest.co.in/public/v2/users/7866998

Send

ParamsAuthorizationHeaders (7)BodyScriptsSettingsCookies

Query Params

	Key	Value	Description	...	Bulk Edit
	Key	Value	Description		

BodyCookies (1)Headers (23)Test Results404 Not Found360 ms1.63 KBSave Response

{ } JSON

PreviewVisualize

```
1 {
2   "message": "Resource not found"
3 }
```

Запрос: DELETE <https://gorest.co.in/public/v2/users/7866997>

DELETE

https://gorest.co.in/public/v2/users/7866997

Send

ParamsAuthorizationHeaders (7)BodyScriptsSettingsCookies

Query Params

	Key	Value	Description	...	Bulk Edit
	Key	Value	Description		

BodyCookies (1)Headers (23)Test Results404 Not Found292 ms1.63 KBSave Response

{ } JSON

PreviewVisualize

```
1 {
2   "message": "Resource not found"
3 }
```

12. Сохраните коллекцию получившихся запросов в Postman.